Detecção de Bola em Partidas de Futebol: Um Pipeline com YOLO e Otimização via Algoritmo Genético

Rafael Bernary de Souza
Gabriel de Oliveira Rêgo

14 de fevereiro de 2025

Este trabalho apresenta um pipeline completo para detecção de bola em partidas de futebol, unindo técnicas de pré-processamento de imagens, algoritmos de detecção baseados na arquitetura YOLO e otimização de parâmetros via Algoritmo Genético (DEAP). O método foi testado em um conjunto de vídeos reais de partidas, alcançando acurácia e velocidade adequadas para aplicações práticas em análise esportiva e suporte de arbitragem.

1 Introdução

A análise automatizada de vídeos esportivos tem ganhado relevância em áreas como análise de desempenho, geração de estatísticas avançadas e apoio à arbitragem [1, 2]. Entre os desafios mais comuns destaca-se a **detecção da bola**, um objeto pequeno e frequentemente oculto por jogadores ou em regiões com pouca iluminação.

Em particular, o futebol é um esporte de grande popularidade mundial. A possibilidade de localizar a bola em cada frame de forma confiável possibilita:

- Geração de mapas de calor de posse de bola e movimentação;
- Estimativas de velocidade e aceleração do objeto;
- Extração de melhores momentos para transmissões televisivas ou plataformas de *streaming*.

Entretanto, a detecção robusta da bola não é trivial, pois envolve variações de iluminação, oclusões frequentes, diferentes cores de gramado e uni-

formes. Técnicas de *visão computacional* vêm sendo aplicadas para enfrentar esses obstáculos [3].

Neste trabalho, propomos um *pipeline* completo que engloba:

1. Coleta e organização de *dataset* de partidas reais,
2. Pré-processamento de imagens com métodos como equalização de histograma, CLAHE e correção gamma,
3. Treinamento da arquitetura **YOLO** (You Only Look Once) para detecção de objetos,
4. Otimização de parâmetros via **Algoritmo Genético** (biblioteca DEAP), incluindo a configuração do pré-processamento e a definição da região de interesse (ROI).

Os resultados apontam ganhos significativos na mAP e na taxa de processamento quando comparados a abordagens sem otimização. Este artigo está estruturado da seguinte forma: a Seção 2 apresenta a fundamentação teórica; a Seção 3 descreve a metodologia, incluindo dataset e pipeline; na Seção 4, discutimos os resultados experimentais; e, por fim, a Seção 6 conclui e propõe trabalhos futuros.

2 Fundamentação Teórica

2.1 Arquitetura YOLO

A família de arquiteturas YOLO (You Only Look Once) introduzida por Redmon et al. [4] revolucionou a detecção de objetos ao propor uma pipeline de inferência em passo único, diferenciando-se de métodos de duas etapas como R-CNN. A evolução do YOLO trouxe variações como YOLOv3, YOLOv4, YOLOv5 e, mais recentemente, YOLOv11. Em linhas gerais, a rede divide a imagem em regiões e, simultaneamente, prevê bounding boxes e probabilidades de classe, atingindo alto *throughput* em GPUs.

2.2 Técnicas de Pré-Processamento de Imagem

Melhorar o *contraste* ou suprimir ruídos é crítico na detecção de objetos pequenos, como a bola. As técnicas empregadas incluem:

- **Equalização de Histograma**: redistribui as intensidades, acentuando contrastes em áreas subexpostas.

- **CLAHE (Contrast Limited Adaptive Histogram Equalization)**: versão adaptativa que segmenta a imagem em pequenos blocos (tiles) e limita a amplificação de contraste para evitar saturação de ruído.
- **Correção Gamma**: ajusta o nível de brilho global via γ, realçando pixels mais escuros ou mais claros.
- **Remoção de Fundo (Background Removal)**: ao segmentar em espaços de cor (HSV, por exemplo), remove-se o céu ou o gramado, destacando a bola.

2.3 Algoritmos Genéticos e Biblioteca DEAP

Algoritmos Genéticos (AG) constituem uma classe de *heurísticas* inspiradas na evolução biológica [5]. Uma população de indivíduos (soluções) é evoluída por seleções, cruzamentos e mutações, buscando maximizar (ou minimizar) uma *fitness function*. Neste projeto, utilizamos a **DEAP** (Distributed Evolutionary Algorithms in Python) para definir:

- Espaço de busca (por exemplo, `preproc` = {*none*, *hist_eq*, *clahe*, *gamma*}, parâmetros de CLAHE, parâmetros de ROI, etc.),
- Função de avaliação (mAP50 e FPS de inferência),
- Operadores de *crossover* e *mutação*.

Dentre os parâmetros otimizados, destaca-se a configuração do ROI (por exemplo, `roi_scale`), que influencia a definição da área da imagem a ser analisada, contribuindo para o balanceamento entre acurácia e velocidade.

3 Metodologia

A metodologia proposta compreende as etapas:

1) **Coleta e Organização do Dataset**,
2) **Pré-Processamento e Criação de Arquivo YAML**,
3) **Treinamento YOLO e Otimização com Algoritmo Genético**,
4) **Validação e Teste em Vídeos Reais**.

3.1 Coleta do Dataset

Partimos de um vídeo completo de uma partida de futebol (90 minutos). Utilizamos o software *Shotcut* para cortar um trecho de interesse com maior movimentação da bola. Esse trecho foi enviado à plataforma **Roboflow**, que extraiu *frames* (a 3 FPS) e permitiu a anotação manual da bola. Após a anotação, o Roboflow gerou um `.zip` contendo pastas `train` e `valid` no formato YOLO. A Tabela 1 descreve o volume de dados obtido.

Tabela 1: Resumo do Dataset de Trechos de Partida.

Conjunto	**Nº de Imagens**	**Proporção**	**Observações**
Train	192	71%	Frames variados, com bola em diversas posições
Valid	49	29%	Usado para aferir a mAP durante o treino

3.2 Pré-Processamento e Arquivo de Configuração

Cada imagem pode passar por transformações como **equalização**, **CLAHE**, **gamma** ou remoção de fundo, conforme parâmetros definidos. Além disso, implementamos a extração de **Região de Interesse (ROI)**, na qual a imagem é recortada com base em um fator de escala (`roi_scale`). Esse parâmetro determina a proporção da imagem original a ser considerada, permitindo focar em áreas com maior probabilidade de conter a bola. Em seguida, criamos um arquivo `.yaml` para o YOLO, especificando caminhos para `train` e `valid`, além do número de classes (`nc=1`) e nome da classe (`names: [‘‘bola’’]`).

3.3 Treinamento YOLO e Otimização por AG

Para cada indivíduo do AG, rodamos um treinamento rápido (por exemplo, 10 épocas) e medimos:

- **mAP50**: *mean Average Precision* com interseção sobre união (IoU) = 0.5;
- **FPS**: taxa de inferência sobre o conjunto `valid`.

A função de aptidão (*fitness*) combina 90% de mAP e 10% de FPS normalizado, buscando equilibrar acurácia e velocidade. Entre os parâmetros

otimizados estão as configurações do pré-processamento (como `gamma_value` e `clahe_clip`) e também o `roi_scale`, que afeta a região analisada. Finalizadas as gerações (por exemplo, 10 gerações com população de 10 indivíduos), seleciona-se o melhor indivíduo, cujos parâmetros são então usados no **treinamento final** (50 épocas).

3.3.1 Uso do Google Colab e GPUs

A fim de acelerar o processo, executamos o GA (com seus diversos treinamentos rápidos) no **Google Colab** usando uma GPU **A100**. Posteriormente, para o treinamento final de 50 épocas, utilizamos uma GPU **L4**, também no Colab. Esse escalonamento permitiu equilibrar custo e desempenho, dado que o GA envolve muitos experimentos curtos, enquanto o treino final demanda estabilidade e menor custo/hora.

3.4 Ferramentas e Implementação

A implementação foi realizada em Python 3, utilizando:

- **Ultralytics/YOLO** (versão mais recente) para detecção,
- **OpenCV** para manipulação de imagens (pré-processamento),
- **DEAP** para o Algoritmo Genético,
- **PyTorch** como backend de treinamento do YOLO.

Também foram criados scripts em Jupyter Notebook para organizar o processo (vide Código 1). A seguir, ilustramos um trecho do código usado para aplicar diferentes estágios de pré-processamento:

```
def apply_pipeline(image, use_hist_eq, use_clahe, clahe_clip, tile_size,
                   use_gamma, gamma_value, use_remove_bg,
                   lower_bg, upper_bg, use_roi, roi_scale):
    processed_img = image.copy()

    # 1) Equalizao de histograma
    if use_hist_eq:
        for i in range(3):
            processed_img[..., i] = cv2.equalizeHist(processed_img[..., i])

    # 2) CLAHE
```

```
    if use_clahe:
        clahe = cv2.createCLAHE(clipLimit=clahe_clip, tileGridSize=
            tile_size)
        for i in range(3):
            processed_img[..., i] = clahe.apply(processed_img[..., i])

    # 3) Correo gamma
    if use_gamma:
        invGamma = 1.0 / gamma_value
        table = np.array([((j / 255.0) ** invGamma) * 255
                          for j in np.arange(256)]).astype("uint8")
        processed_img = cv2.LUT(processed_img, table)

    # 4) Remoo de fundo
    if use_remove_bg:
        lower = np.array(lower_bg, dtype="uint8")
        upper = np.array(upper_bg, dtype="uint8")
        mask = cv2.inRange(processed_img, lower, upper)
        processed_img[mask > 0] = 0

    # 5) Extrao de ROI
    if use_roi:
        h, w = processed_img.shape[:2]
        new_h, new_w = int(h/roi_scale), int(w/roi_scale)
        start_y = (h - new_h) // 2
        start_x = (w - new_w) // 2
        processed_img = processed_img[start_y:start_y+new_h, start_x:
            start_x+new_w]

    return processed_img
```

Listing 1: Trecho de código para pipeline de pré-processamento.

4 Resultados

4.1 Otimização via Algoritmo Genético (GA)

Durante a fase de otimização, foram rodadas 10 gerações, com 10 indivíduos cada, totalizando 100 experimentos de treinamento rápido. Todos os logs foram salvos em um CSV denominado `ga_detailed_evaluations(1).csv`. A Tabela 2 mostra algumas linhas de exemplo:

Ao final, um parâmetro "ótimo" emergiu (por exem-

Tabela 2: Exemplo de linhas extraídas do arquivo `ga_detailed_evaluations(1).csv`.

preproc	remove_bg	clahe_clip	gamma_value	mAP50	fps	score
gamma	False	1.87	0.82	0.47	59.31	0.62
none	False	3.52	1.28	0.47	60.91	0.63
clahe	True	3.89	1.21	0.47	60.88	0.63
...	...	...	...	...	...	...
gamma	True	2.83	1.21	0.47	61.80	0.63

plo: `preproc=‘‘gamma’’`, `remove_bg=True`, `clahe_clip=2.83`, `gamma_value=1.21`, `tile_size=(16,16)`, `roi_scale=4.27`), com `mAP50=0.47` e `fps~61`, gerando um `score~0.63` nos treinamentos rápidos. Estes detalhes foram salvos em `best_preproc_params.json`, incluindo também o `training_time` e a configuração final de ROI.

4.2 Treinamento Final e Avaliação Ampliada

Com a melhor configuração, executou-se o treinamento final por 50 épocas na **GPU L4**. Observou-se que a mAP subiu significativamente para valores acima de 0.80, devido ao maior número de épocas.

4.3 Avaliação em Vídeo

Foram realizados testes com um vídeo de 1 minuto de partida, onde medimos a quantidade de detecções de bola e a taxa de processamento (FPS) para diferentes configurações do pipeline. A Tabela 3 resume os resultados obtidos:

Configuração	Detecções de Bola	FPS Estimado
YOLO v11 small (original, `output_no_preproc.mp4`)	724	21.48
YOLO v11 final (original, `output_no_preproc.mp4`)	1533	25.66
YOLO v11 final (pré-processamento, `output_preproc.mp4`)	760	18.09
YOLO v11 final (pré-processamento com ROI, `output_preproc_roi.mp4`)	782	12.70
YOLO v11 small (pré-processamento, `output_preproc.mp4`)	1406	17.81
YOLO v11 small (pré-processamento com ROI, `output_preproc_roi.mp4`)	1582	15.42

Tabela 3: Comparação dos resultados obtidos para diferentes configurações.

Observa-se que o modelo **YOLO v11 final** apresentou desempenho superior em termos de detecções quando comparado à versão **YOLO v11 small**.

Ademais, os testes com pré-processamento (total ou com ROI) reduziram a taxa de FPS, especialmente na configuração com ROI.

5 Discussão e Desafios

Durante o desenvolvimento deste trabalho, alguns desafios se destacaram:

- **Equilíbrio entre Acurácia e Velocidade:** O treinamento final com o YOLO v11 final, utilizando parâmetros otimizados, elevou a contagem de detecções de 724 para 1533. Entretanto, a aplicação de técnicas de pré-processamento — especialmente com ROI — reduziu significativamente a taxa de FPS. Embora a extração de ROI ajude a reduzir falsos positivos ao focar em áreas de interesse, esse recorte pode aumentar o tempo de processamento, o que deve ser considerado em aplicações de tempo real.
- **Falsos Positivos:** As experimentações com o YOLO v11 small associadas ao pré-processamento geraram um elevado número de falsos positivos, comprometendo a confiabilidade da detecção.
- **Custo Computacional:** A execução dos múltiplos treinamentos rápidos necessários para a otimização via Algoritmo Genético demanda considerável poder computacional, justificando o uso de GPUs de alta performance (como a A100) para acelerar o processo.

Esses pontos evidenciam a necessidade de futuros aprimoramentos, tais como a integração de métodos de rastreamento para melhorar a continuidade da detecção, ajustes finos dos parâmetros de pré-processamento (inclusive o ROI) e a investigação de arquiteturas que possam oferecer um melhor trade-off entre acurácia e velocidade.

6 Conclusão

Este trabalho apresentou um pipeline completo para a detecção da bola em partidas de futebol, integrando técnicas de pré-processamento, treinamento da arquitetura YOLO e otimização de parâmetros por meio de Algoritmos Genéticos. Os principais achados foram:

- A utilização do modelo **YOLO v11 final** treinado com parâmetros otimizados resultou em um aumento expressivo na detecção de bola (de 724 para 1533 detecções) quando comparado ao modelo base YOLO v11 small.
- A aplicação de pré-processamento, embora contribua para a redução dos falsos positivos, impactou negativamente na taxa de FPS, especialmente quando combinada com a extração de ROI.
- Os testes com o YOLO v11 small associados ao pré-processamento apresentaram altos índices de falsos positivos, não sendo considerados para a análise final.

Em síntese, a abordagem proposta demonstrou eficácia na melhoria da acurácia da detecção, mas evidenciou um trade-off relevante entre desempenho e velocidade de processamento. Como trabalhos futuros, recomenda-se a integração de técnicas de rastreamento, o ajuste fino dos parâmetros de pré-processamento (incluindo o ROI) e a experimentação com arquiteturas alternativas para aprimorar ainda mais o desempenho do sistema em cenários reais.

Referências

[1] Board Consulting. FIFA's Technology Innovations in Football, 2018. Acesso em: 01 fev. 2025.

[2] Andrews, T., Smith, R. Sports AI Revolution: Real-time Analysis and Data Mining. *Journal of Sports Technology*, 15(4), 2022.

[3] Choi, Y., Kim, S. Ball Detection in Soccer Videos using CNN-based Approach. In: *International Conference on Image Processing*, 2019.

[4] Redmon, J., Farhadi, A. YOLO9000: Better, Faster, Stronger. *CVPR*, 2017.

[5] Goldberg, D. E. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, 1989.